



FACIAL EMOTION RECOGNITION

Group 1

DATASET

FER-2013



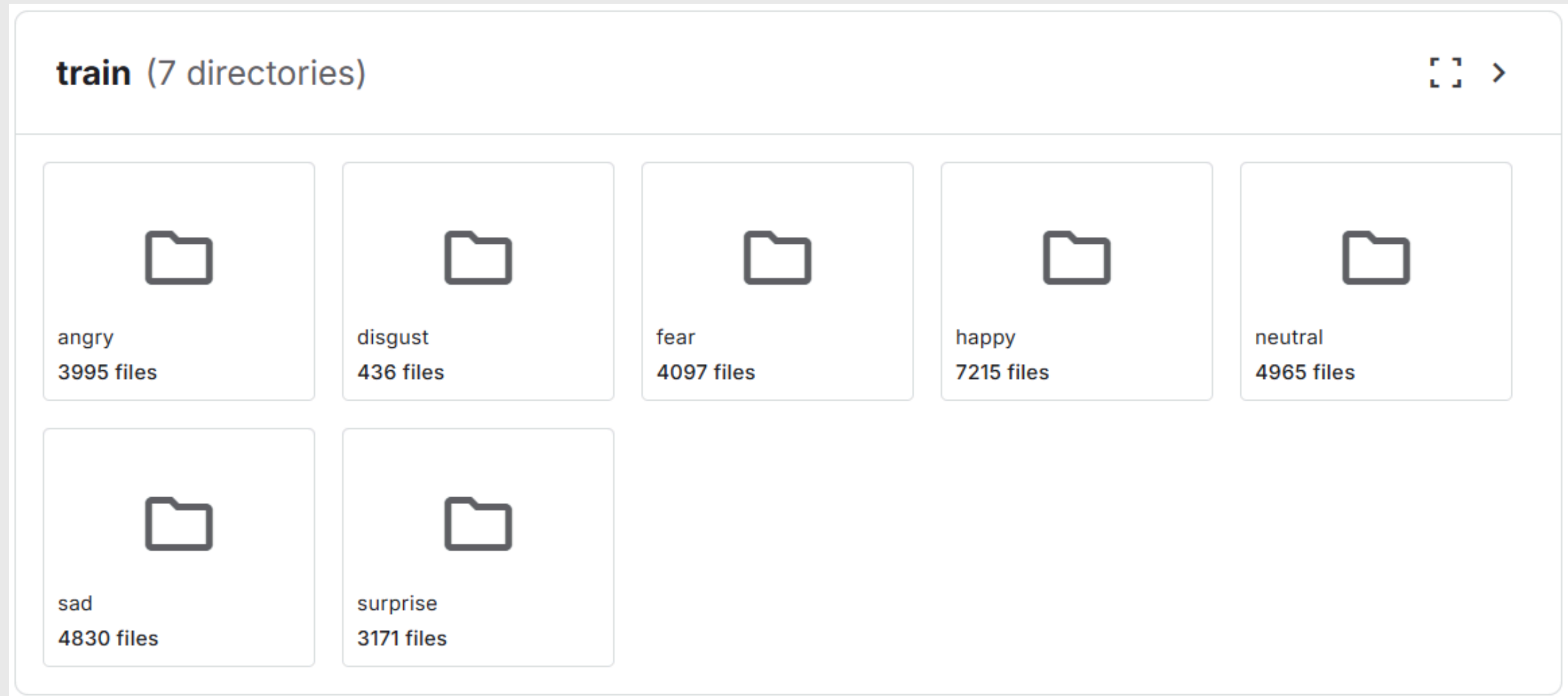
FER-2013

Learn facial expressions from an image

[k kaggle.com](https://www.kaggle.com/datasets/fer2013)

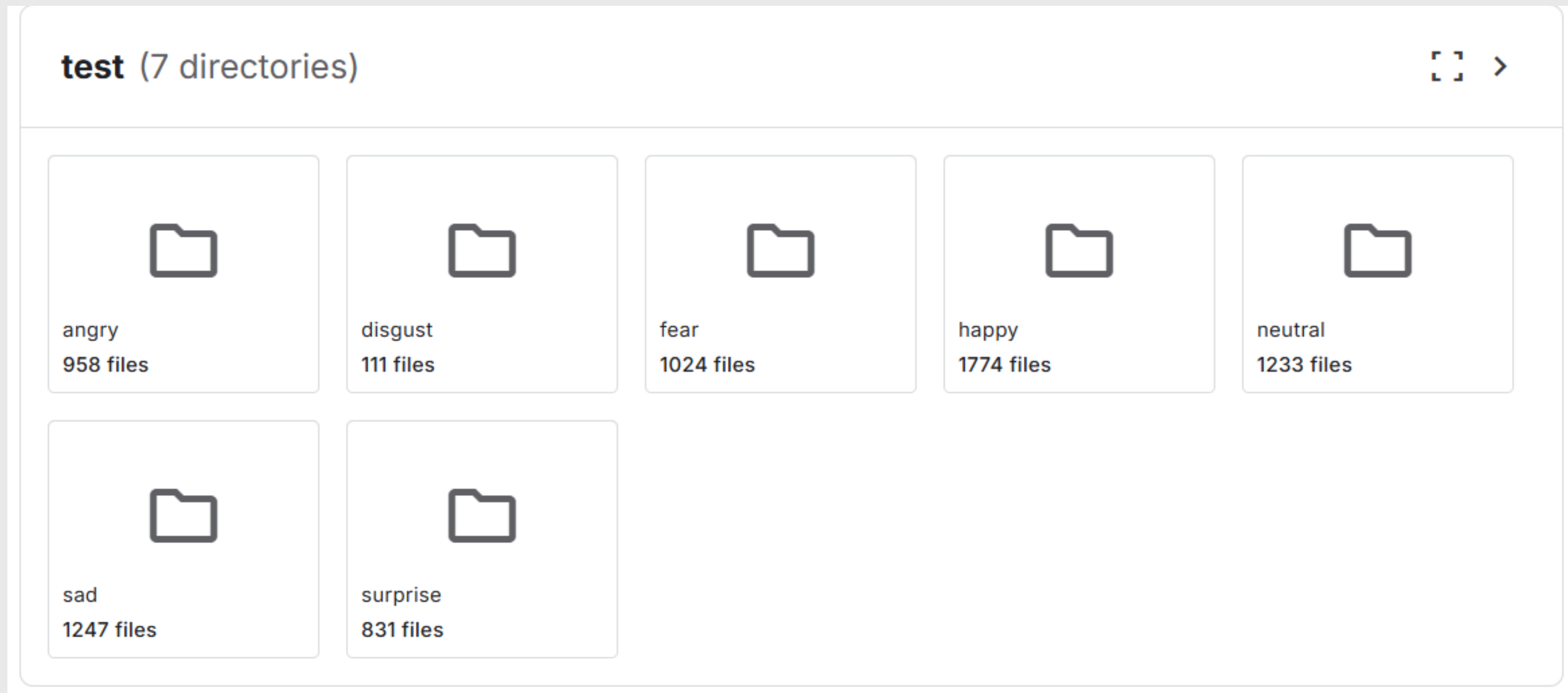
DATASET

Train folder



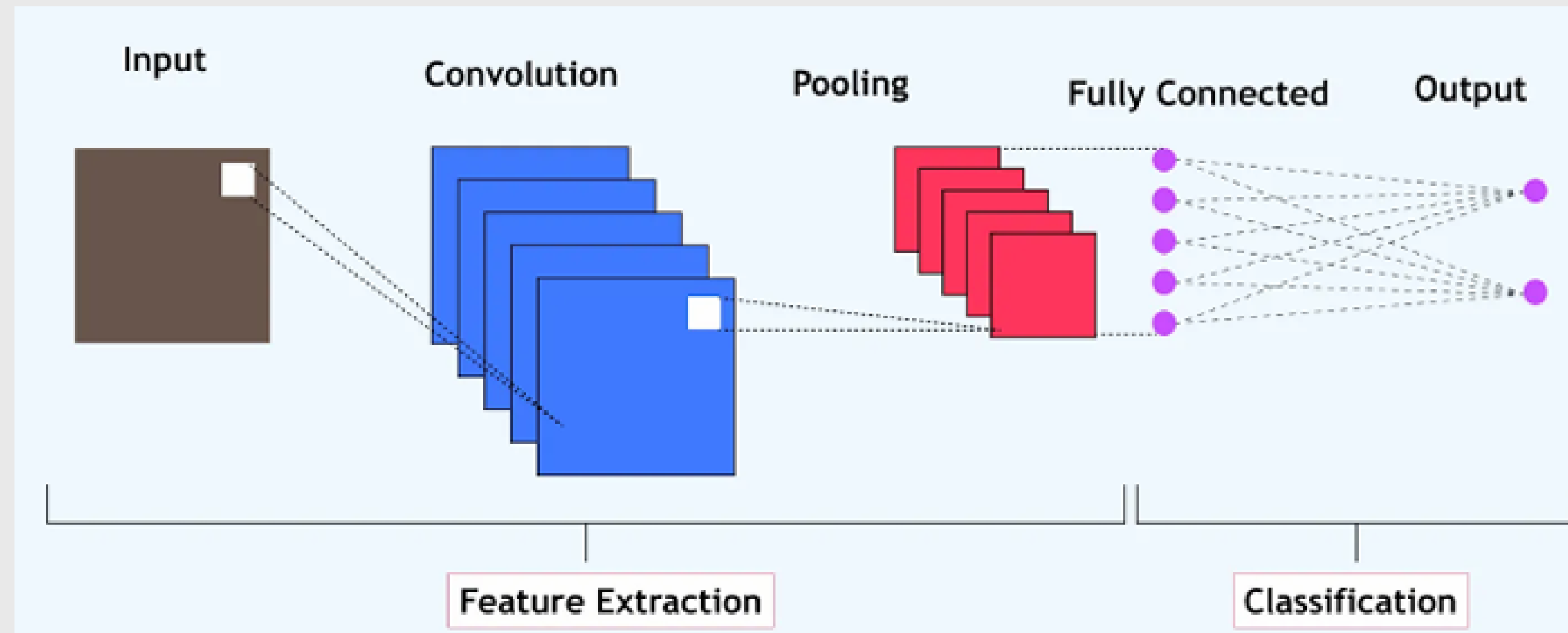
DATASET

Test folder



INITIAL APPROACH

Custom CNN





PROJECT GOALS

1. Design a Deep Learning model that classifies the images according to facial emotions in the dataset.
2. Validation Accuracy: 70~80%

1. IMPORT LIBRARIES

```
import tensorflow as tf  
from tensorflow import keras  
import numpy as np  
import matplotlib.pyplot as plt  
import cv2
```

Used libraries include:

Tensorflow: for model building and training

cv2 (opencv): images manipulation (mainly resizing, color conversion)

Numpy: Matrix manipulation

Matplotlib: use to display the result using graphs and plots

2. DATASET PULL

```
!pip install gdown  
!gdown "https://drive.google.com/uc?id=1JB9-bIhu7I3jtoYNZi9gSpbfrMtBaVZ4"  
!unzip fer2013.zip
```

Use gdown (alternative to wget) to get the dataset from Google Drive and unzip it

2. DATASET PULL

Prepare the dataset:

- A validation/train split of 20/80
- Image size (48x48) with 1 channel grayscale (actual dataset)

```
train_datagen = keras.preprocessing.image.ImageDataGenerator(  
    rescale=1./255,  
    validation_split=0.2  
)  
# create training data  
train_generator = train_datagen.flow_from_directory(  
    "train",  
    target_size=(48,48),      #image already 48x48  
    color_mode='grayscale',  
    batch_size=64,  
    class_mode='categorical',  #read from the directory folder "train" -> each sub-folder in "train" will recognize as a class  
    subset='training'         # use the training subset only ( 0.8 of all instances in "train")  
)  
#links random instance to a batch then when a batch is picked -> perform preprocess then put in to the training (i think)  
  
# create validation data  
validation_generator = train_datagen.flow_from_directory(  
    "train",  
    target_size=(48,48),  
    color_mode='grayscale',  
    batch_size=64,  
    class_mode='categorical',  
    subset='validation'  
)  
  
#create testing data  
test_datagen = keras.preprocessing.image.ImageDataGenerator(  
    rescale=1./255,  
)  
test_generator = test_datagen.flow_from_directory(  
    "test",  
    target_size=(48,48),  
    color_mode='grayscale',  
    batch_size=64,  
    class_mode='categorical',  
)
```

3. MODEL DEFINITION

Model 1: The original

Model features:

- 3 pairs of convolution + MaxPooling
- Flatten → Dense → Dropout → Dense.

Results:

- Training accuracy: 0.6567
- Validation accuracy: 0.5558

```
#CNN layers
model = keras.models.Sequential([

    keras.layers.Conv2D( 32, (3,3), activation='relu', input_shape=(48,48,1)),
    keras.layers.MaxPooling2D((2,2)),
    keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(7, activation='softmax')

])
```

4. MODEL COMPILE & TRAINNING

```
# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train the model
history = model.fit(train_generator, epochs=15, validation_data=validation_generator)
```



COMPARISION BETWEEN ALTERNATIVE STRUCTURE

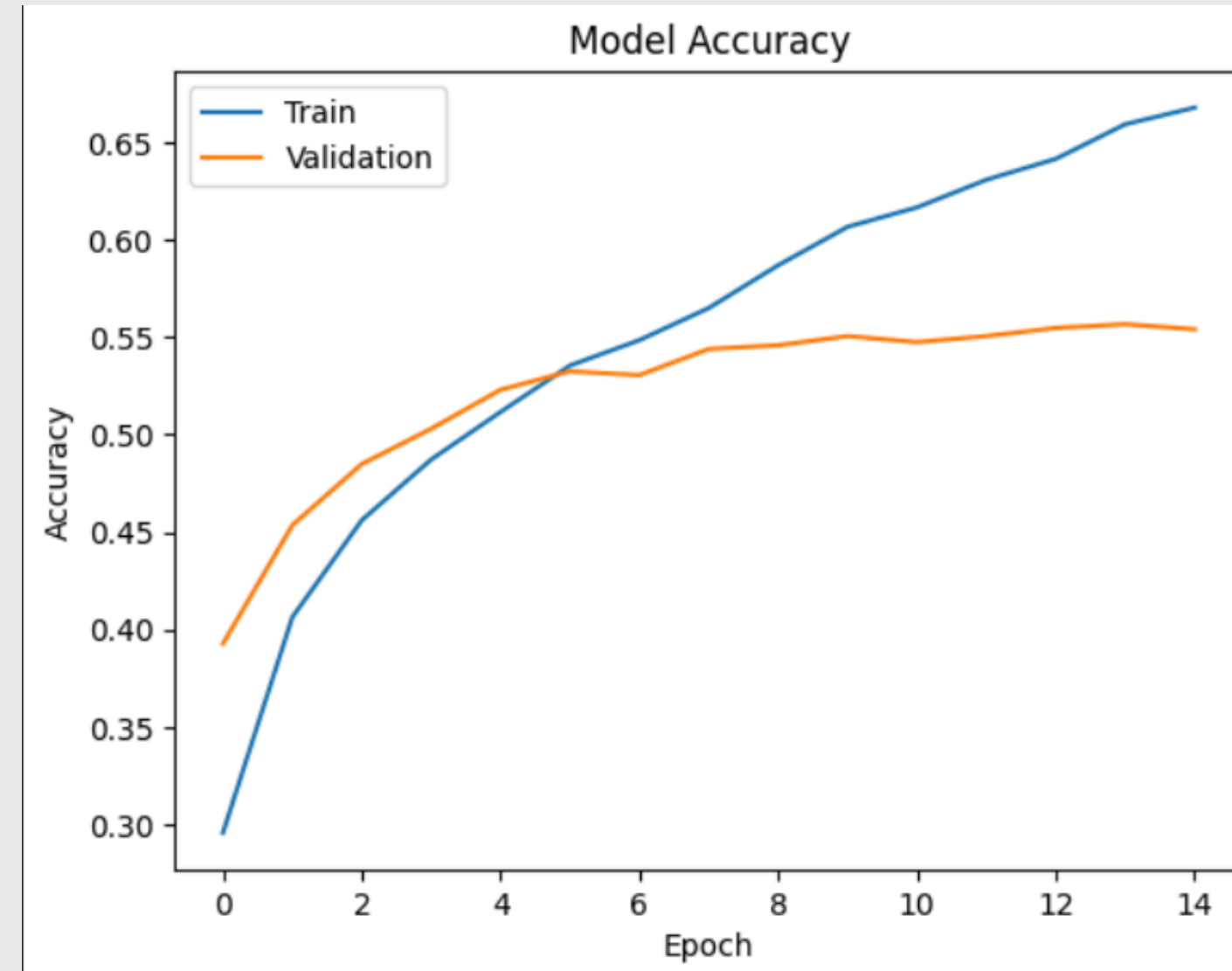
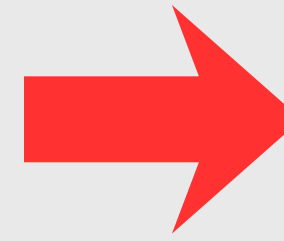
FIRST PROTOTYPE

```
#CNN layers
model = keras.models.Sequential([

    keras.layers.Conv2D( 32, (3,3), activation='relu', input_shape=(48,48,1)),
    keras.layers.MaxPooling2D((2,2)),
    keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(7, activation='softmax')

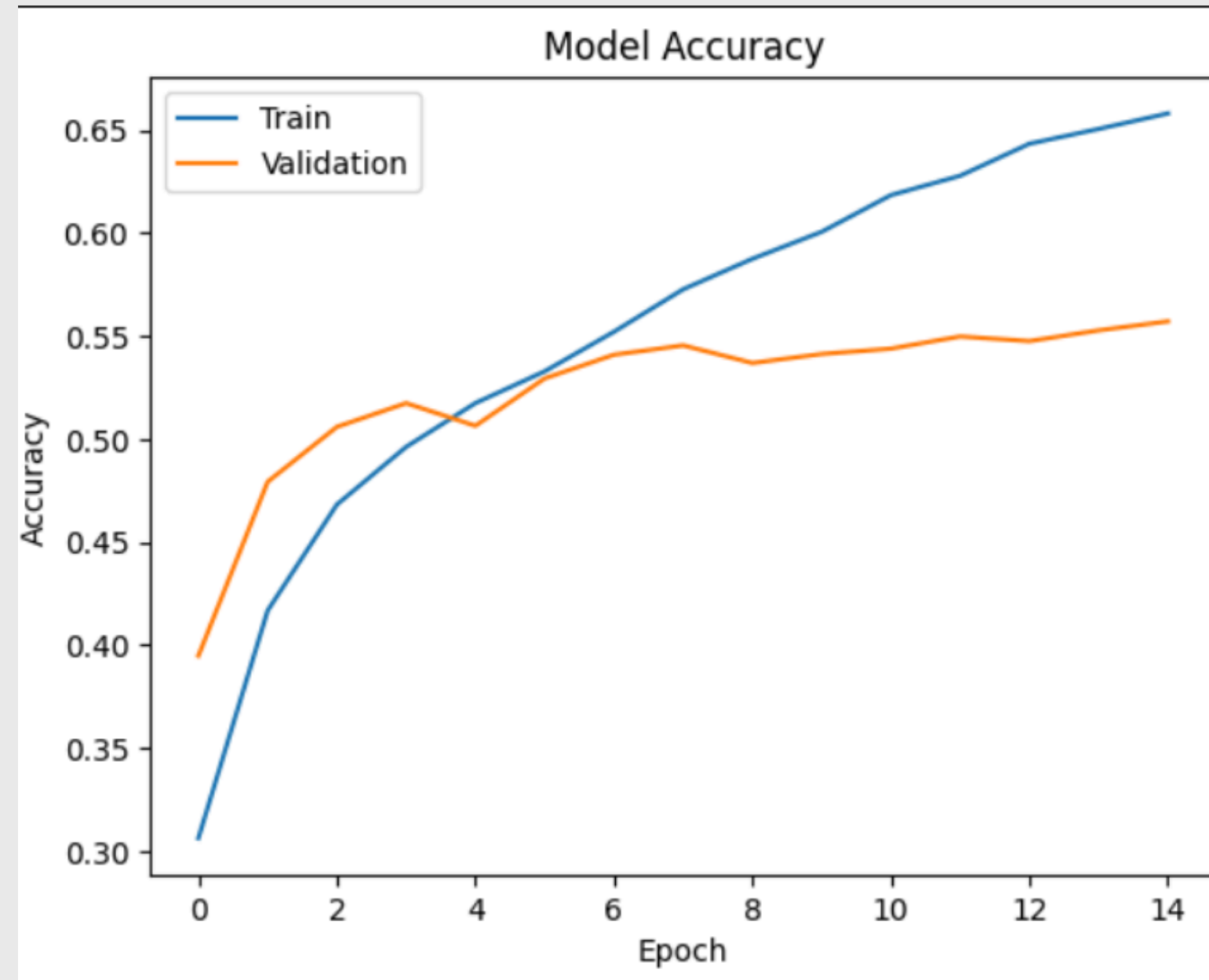
])
```

Batch size = 64



1st_Alternatives: batch size = 16

Modify from the “first prototype” by changing batch size to 16



→ Almost the same as batch size = 64

2nd_Alternatives: Adding batch normalization and some more drop out

```
#CNN layers
model = keras.models.Sequential([

    keras.layers.Conv2D( 32, (3,3), activation='relu', input_shape=(48,48,1)),
    keras.layers.BatchNormalization(), ← new layer
    keras.layers.MaxPooling2D((2,2)),
    keras.layers.Dropout(0.1), ← new layer

    keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    keras.layers.BatchNormalization(), ← new layer
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Dropout(0.1), ← new layer

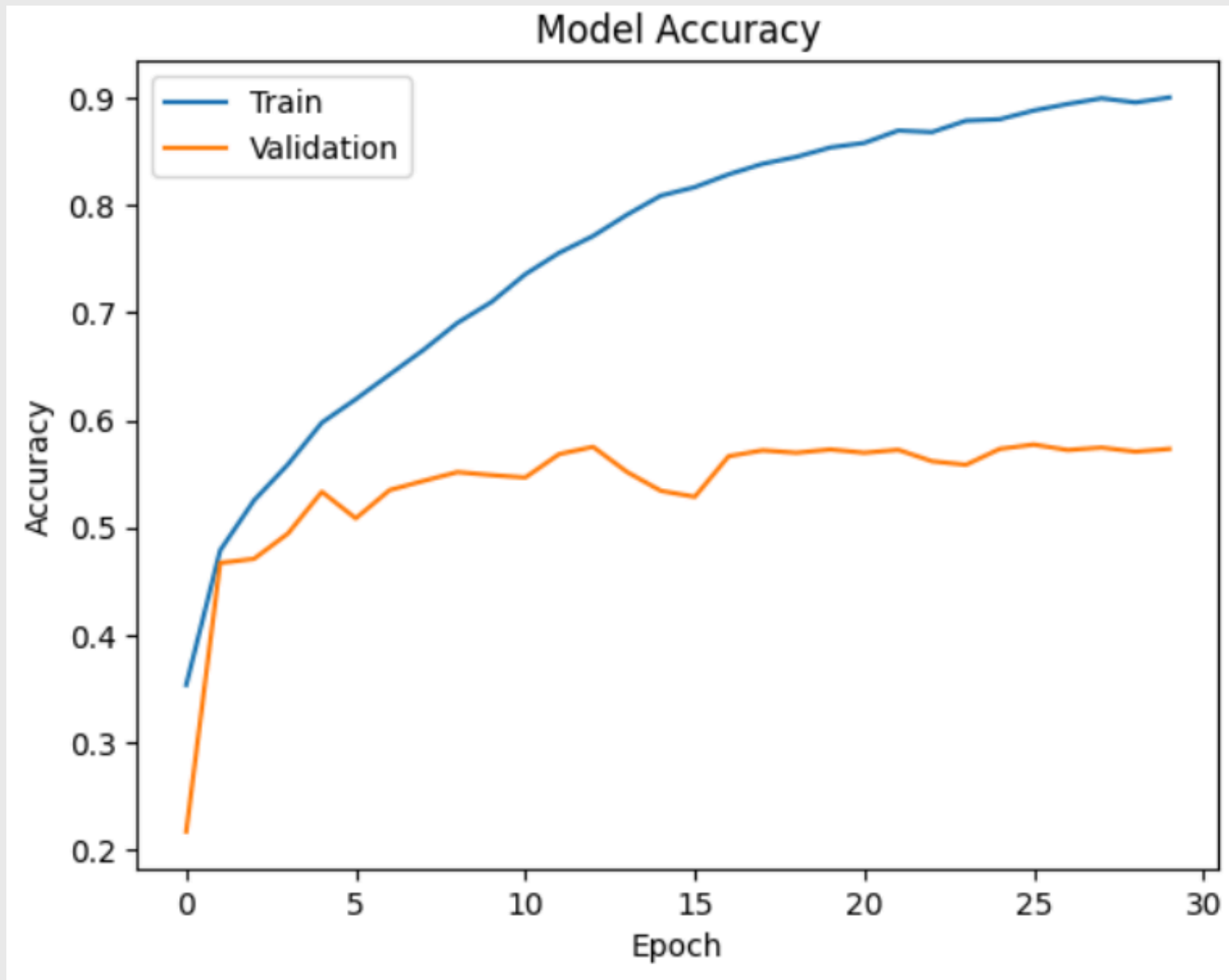
    keras.layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
    keras.layers.BatchNormalization(), ← new layer
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Dropout(0.1), ← new layer

    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.3), ← drop out reduce from 0.5 to 0.3
    keras.layers.Dense(7, activation='softmax')

])
```

2nd_Alternatives: Adding batch normalization and some more drop out

Result:



```
h 23/30
359 ----- 151s 421ms/step - accuracy: 0.8679 - loss: 0.3590 - val_accuracy: 0.5617
h 24/30
359 ----- 199s 413ms/step - accuracy: 0.8785 - loss: 0.3359 - val_accuracy: 0.5584
h 25/30
359 ----- 153s 426ms/step - accuracy: 0.8799 - loss: 0.3317 - val_accuracy: 0.5732
h 26/30
359 ----- 145s 405ms/step - accuracy: 0.8880 - loss: 0.3114 - val_accuracy: 0.5773
h 27/30
359 ----- 145s 404ms/step - accuracy: 0.8941 - loss: 0.2918 - val_accuracy: 0.5722
h 28/30
359 ----- 147s 409ms/step - accuracy: 0.8994 - loss: 0.2849 - val_accuracy: 0.5745
h 29/30
359 ----- 151s 422ms/step - accuracy: 0.8955 - loss: 0.2914 - val_accuracy: 0.5706
h 30/30
359 ----- 151s 420ms/step - accuracy: 0.9001 - loss: 0.2721 - val_accuracy: 0.5732
```

Validation accuracy slightly improve but severe overfitting

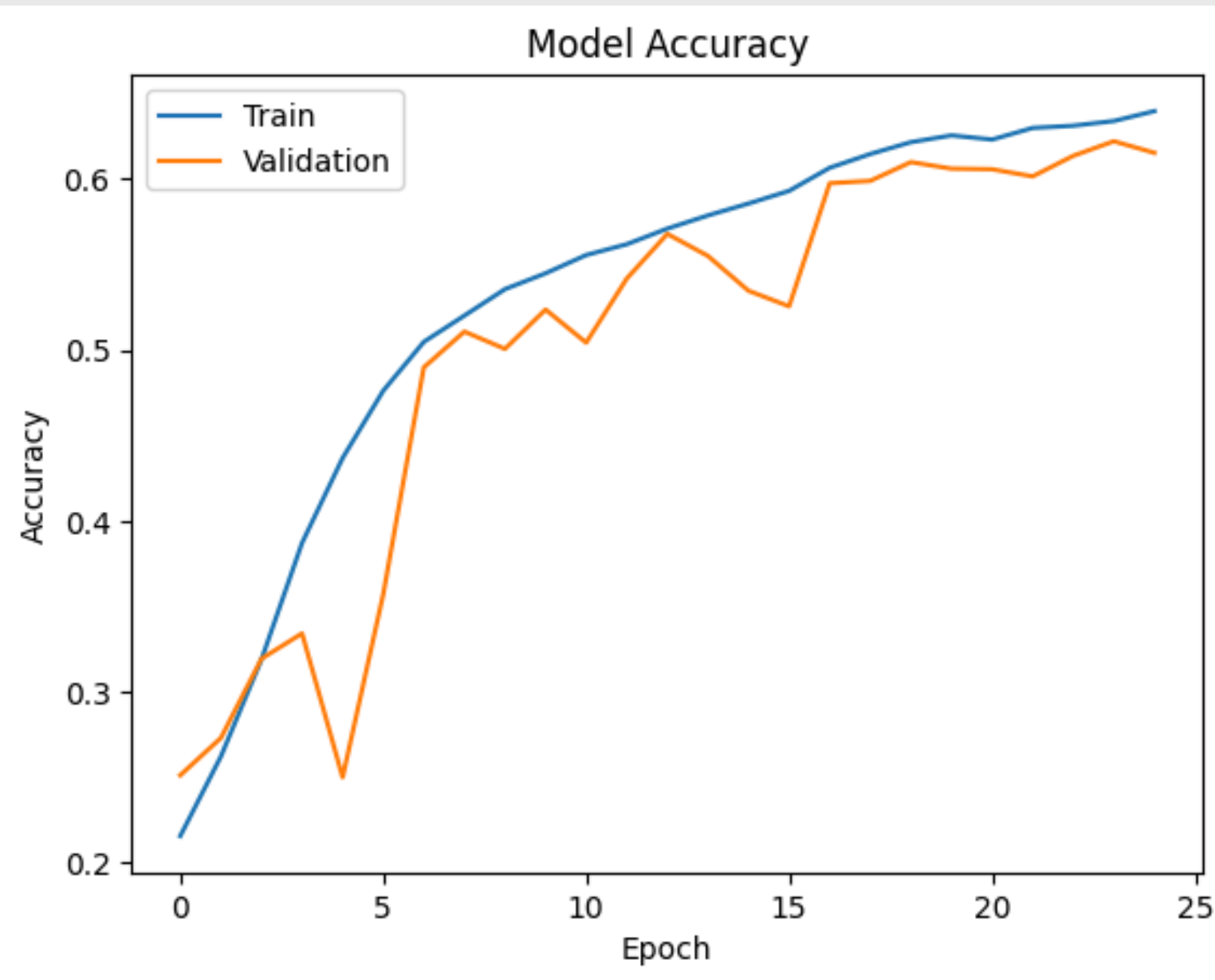
3rd_Alternatives: 2 Convolution layers before Maxpooling layer

```
keras.layers.Conv2D( 32, (3,3), activation='relu',padding='same', input_shape=(48,48,1)),
keras.layers.BatchNormalization(),
keras.layers.Activation('relu'),
keras.layers.Conv2D( 32, (3,3), activation='relu', padding='same', input_shape=(48,48,1)),
keras.layers.BatchNormalization(),
keras.layers.Activation('relu'),
keras.layers.MaxPooling2D((2,2)),

keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
keras.layers.BatchNormalization(),
keras.layers.Activation('relu'),
keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same'),← Adding 1 more Convolution layer
keras.layers.BatchNormalization(),
keras.layers.Activation('relu'),
keras.layers.MaxPooling2D((2, 2)),
```

3rd_Alternatives: Adding 1 more Convolution layer before Maxpooling layer

Result:



Epoch 15/20										
359/359	41s	65ms/step	-	accuracy: 0.6470	-	loss: 0.9588	-	val_accuracy: 0.6239	-	val_loss: 1.0069
Epoch 16/20										
359/359	23s	65ms/step	-	accuracy: 0.6486	-	loss: 0.9539	-	val_accuracy: 0.6210	-	val_loss: 1.0079
Epoch 17/20										
359/359	23s	65ms/step	-	accuracy: 0.6480	-	loss: 0.9565	-	val_accuracy: 0.6253	-	val_loss: 1.0123
Epoch 18/20										
359/359	23s	65ms/step	-	accuracy: 0.6460	-	loss: 0.9594	-	val_accuracy: 0.6208	-	val_loss: 1.0042
Epoch 19/20										
359/359	24s	67ms/step	-	accuracy: 0.6503	-	loss: 0.9475	-	val_accuracy: 0.6180	-	val_loss: 1.0140
Epoch 20/20										
359/359	24s	66ms/step	-	accuracy: 0.6444	-	loss: 0.9596	-	val_accuracy: 0.6208	-	val_loss: 1.0159

Validation accuracy slightly increase to 62%

4th_Alternatives: Apply transfer learning from the model "ConvNeXtTiny"

```
# Unfreeze strategy: unfreeze only the final architectural block
backbone.trainable = True
for layer in backbone.layers: ← Convoluton layer replaced
    # 'stage4' contains the final blocks in Keras ConvNeXt implementations
    if 'stage4' not in layer.name:
        layer.trainable = False

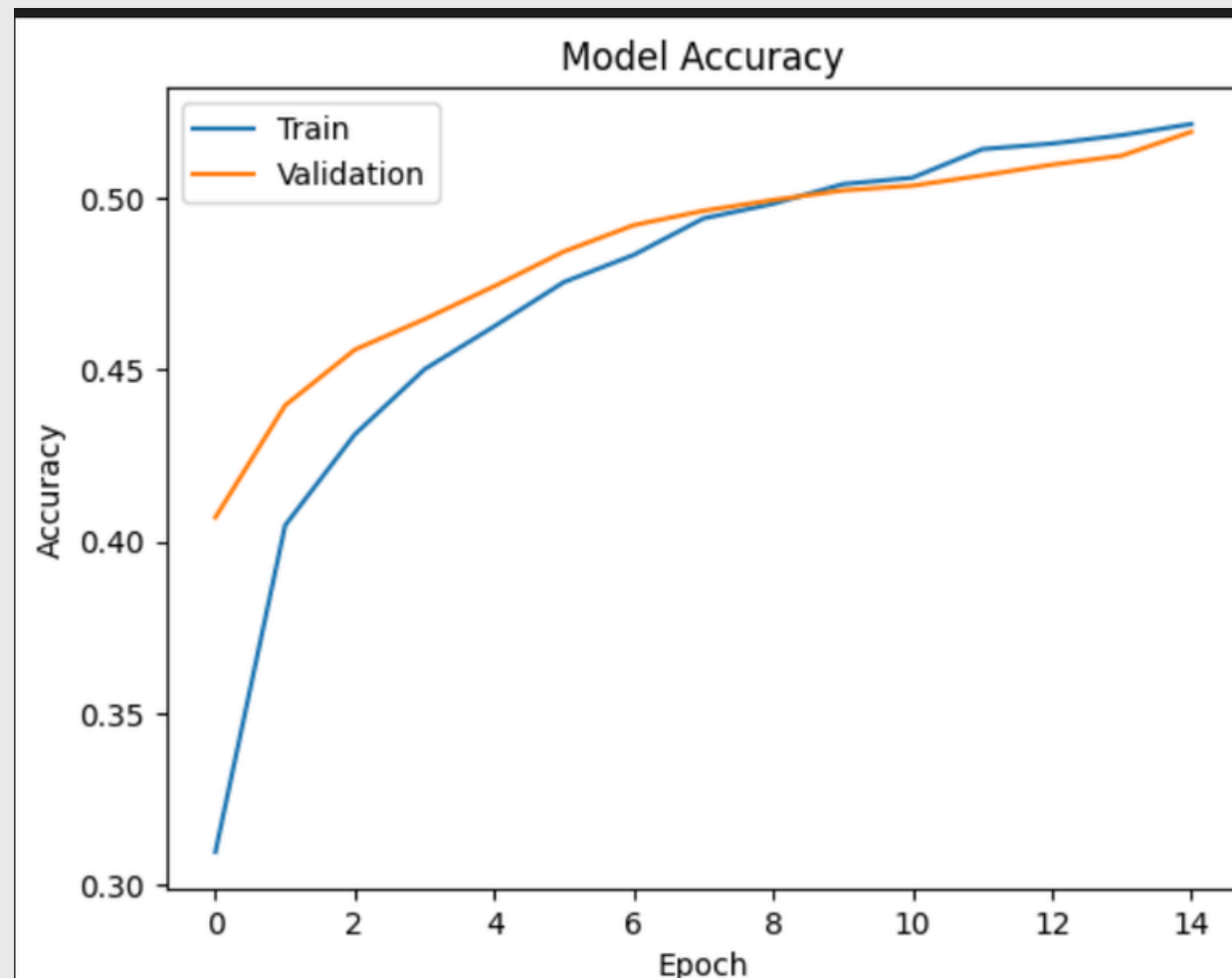
# Pass x into backbone (let Keras handle training vs inference mode)
x = backbone(x)

# Classification Head
x = keras.layers.GlobalAveragePooling2D()(x)
x = keras.layers.Dense(128, activation='relu')(x)
x = keras.layers.Dropout(0.5)(x)
outputs = keras.layers.Dense(7, activation='softmax')(x)

# Create the final model
model = keras.Model(inputs=inputs, outputs=outputs)
[5]
```

4th_Alternatives: Apply transfer learning from the model "ConvNeXtTiny"

Result:



```
Epoch 10/15
359/359 ————— 133s 371ms/step - accuracy: 0.5040 - loss: 1.3324 - val_accuracy: 0.5022 - val_loss: 1.3137
Epoch 11/15
359/359 ————— 133s 370ms/step - accuracy: 0.5059 - loss: 1.3187 - val_accuracy: 0.5036 - val_loss: 1.3052
Epoch 12/15
359/359 ————— 132s 368ms/step - accuracy: 0.5142 - loss: 1.3028 - val_accuracy: 0.5065 - val_loss: 1.2964
Epoch 13/15
...
Epoch 14/15
359/359 ————— 134s 375ms/step - accuracy: 0.5182 - loss: 1.2848 - val_accuracy: 0.5123 - val_loss: 1.2805
Epoch 15/15
359/359 ————— 132s 369ms/step - accuracy: 0.5215 - loss: 1.2762 - val_accuracy: 0.5192 - val_loss: 1.2752
```



FUTURE PLANS

- Apply adaptive learning rate and stop training to prevent overfitting (original approach)
- Look into Pytorch and Hugging Face model (transfer learning approach)



THANK YOU

